

Semana 04

Multiplicación, división y punto flotante

IC3101: Arquitectura de computadores

Objetivos de la primera sesión

Comprender los fundamentos y diseño de algoritmos de hardware para aritmética avanzada:

- **Multiplicación de enteros sin signo:** Comprender el mecanismo iterativo de sumas y desplazamientos y el rol de los registros combinados.
- **Algoritmo de Booth:** Dominar el tratamiento directo de enteros con signo en complemento a dos mediante recodificación de bits contiguos.
- **División no restauradora:** Analizar la reducción de ciclos frente al método restaurador al posponer correcciones del residuo.
- **Fundamentos de punto flotante:** Entender la motivación de la coma flotante y la descomposición del estándar IEEE 754.
- **Casos especiales y anomalías numéricas:** Identificar desbordamientos, subnormales, infinitos, NaN y errores de absorción y cancelación.

Multiplicación de enteros sin signo

La multiplicación binaria en hardware replica el principio de sumas y desplazamientos sucesivos del cálculo manual, operando bit a bit sobre el multiplicador.

Ejemplo en base 2:

```

  1 0 1 1   (11 decimal: Multiplicando)
x 1 1 0 1   (13 decimal: Multiplicador)
-----
  1 0 1 1   (Bit 0 es 1: suma multiplicando)
 0 0 0 0 .   (Bit 1 es 0: solo desplazamiento)
 1 0 1 1 . .   (Bit 2 es 1: suma multiplicando)
 1 0 1 1 . . .   (Bit 3 es 1: suma multiplicando)
-----
1 0 0 0 1 1 1 1 (143 decimal: Producto de 2n bits)
```

Regla de evaluación por ciclo:

Si el bit menos significativo del multiplicador es 1, se suma el multiplicando al acumulador. Si el bit es 0, no se realiza suma alguna.

Desplazamiento a la derecha:

Tamaño del producto:

Organización del hardware de multiplicación

Registro M (Multiplicando)

Contiene el multiplicando de n bits de forma constante durante todo el proceso.

Registro A (Acumulador)

Inicializado en 0. Almacena la parte alta del producto parcial y recibe el resultado del sumador.

Registro Q (Multiplicador)

Almacena el multiplicador de n bits. Al desplazarse, la parte baja del producto entra en este registro.

Estructura de registros combinados:

Registro C (1b) + Registro A (n bits) + Registro Q (n bits)

C

Acumulador A (n bits)

Multiplicador Q (n bits)

← Acarreo del sumador

→ Desplazamiento lógico conjunto a la derecha →

Inspección de Q_0 →

Ciclo de control: En cada paso, la lógica de control inspecciona el bit Q_0 . Si $Q_0 = 1$, la ALU suma $A + M$ y guarda el resultado en C y A. Seguidamente, toda la estructura $[C, A, Q]$ se desplaza un bit a la derecha y el contador de ciclos disminuye en uno.

Multiplicación con enteros con signo

Cuando los operandos están expresados en complemento a dos, el algoritmo sin signo ordinario falla debido al peso aritmético negativo del bit más significativo.

Por qué falla el algoritmo directo:

En complemento a dos de 4 bits, el número 1101_2 representa el valor -3, dado que el bit superior tiene peso $-2^3 = -8$. Si el hardware lo trata como entero sin signo (13), el producto resultante será totalmente incorrecto.

Estrategia ingenua:

Convertir ambos operandos a magnitudes positivas calculando su complemento a dos previo, multiplicar como enteros sin signo y luego aplicar el signo adecuado al resultado final.

Desventajas de la estrategia ingenua:

Requiere etapas adicionales de comprobación de signo, cálculo de complemento a dos previo y posterior, aumentando la latencia y la complejidad del circuito de control.

La solución óptima: Algoritmo de Booth

Permite operar directamente con números positivos y negativos en complemento a dos sin conversiones preliminares, acelerando además la ejecución mediante recodificación de bits.

Fundamento del algoritmo de Booth

El algoritmo de Booth se fundamenta en que una secuencia continua de unos binarios puede reemplazarse por una única resta en el inicio del bloque y una única suma al final del bloque.

Identidad matemática:

$$011110_2 = 2^4 + 2^3 + 2^2 + 2^1 = 30_{10}$$

$$011110_2 = 2^5 - 2^1 = 32 - 2 = 30_{10}$$

Cuatro sumas sucesivas se convierten en solo una resta y una suma.

Recodificación de pares de bits:

Se inspecciona el bit actual del multiplicador (Q_0) y el bit previo inmediatamente descartado a la derecha (Q_{-1}).

Par [Q_0 , Q_{-1}]	Transición de bits	Acción en la ALU	Explicación conceptual
1 0	Inicio de bloque de unos	$A \leftarrow A - M$	Se resta el multiplicando del acumulador
0 1	Fin de bloque de unos	$A \leftarrow A + M$	Se suma el multiplicando al acumulador
0 0	Cadena continua de ceros		
1 1	Interior de bloque de unos		

Flujo operativo del algoritmo de Booth

1. Inicialización:

$A = 0$, $Q_{-1} = 0$, M = multiplicando, Q = multiplicador, Contador = n .

2. Decisión aritmética:

Evaluar $[Q_0, Q_{-1}]$ y ejecutar $A \leftarrow A - M$, $A \leftarrow A + M$ o mantener A intacto.

3. Desplazamiento aritmético:

Desplazar a la derecha la cadena $[A, Q, Q_{-1}]$ conservando el bit de signo de A .

4. Condición de término:

Detalle crítico: Desplazamiento aritmético

desplazamiento aritmético a la derecha

Preservación del signo:

$$A_{n-1} \rightarrow A_{n-1} \ A_{n-2} \dots A_0 \rightarrow Q_{n-1} \dots Q_0 \rightarrow Q_{-1}$$

Esto asegura que los valores negativos mantengan su representación matemática correcta en complemento a dos durante cada iteración.

Traza de cálculo con algoritmo de Booth

Multiplicación de $7 \times (-3) = -21$ en 4 bits: Multiplicando $M = 0111_2$ (7), $-M = 1001_2$ (-7), Multiplicador $Q = 1101_2$ (-3).

Paso / Ciclo	Operación realizada	Acumulador A	Registro Q	Q_{-1}	Contador
Inicio	Valores iniciales	0000	1101	0	4
Ciclo 1	$Q_0Q_{-1} = 10 \rightarrow A \leftarrow A - M$, Desp. aritmético	1100	1110	1	3
Ciclo 2	$Q_0Q_{-1} = 01 \rightarrow A \leftarrow A + M$, Desp. aritmético	0001	1111	0	2
Ciclo 3	$Q_0Q_{-1} = 10 \rightarrow A \leftarrow A - M$, Desp. aritmético	1101	0111	1	1
Ciclo 4	$Q_0Q_{-1} = 11 \rightarrow$ Solo desp. aritmético	1110	1011	1	0

Resultado final combinado $[A, Q]$: $11101011_2 \Rightarrow -128 + 64 + 32 + 8 + 2 + 1 = -21_{10}$

División de enteros en hardware

La división es la operación inversa de la multiplicación, donde para un dividendo D y un divisor V , se determinan un cociente Q y un residuo R que satisfacen la relación fundamental:

$$D = Q \times V + R \quad (\text{con } 0 \leq R < V)$$

Esquema de registros:

El dividendo se carga inicialmente en el registro combinado $[A, Q]$, mientras que el divisor se aloja en el registro M . El cociente final se acumula en Q y el residuo final queda en A .

Mecanismo secuencial:

1. Desplazar la combinación $[A, Q]$ un bit hacia la izquierda.
2. Restar el divisor del acumulador: $A \leftarrow A - M$.
3. Evaluar el signo del residuo parcial obtenido en A .
4. Fijar el bit del cociente Q_0 en 1 si el resultado fue positivo, o en 0 si fue negativo.

Manejo de excepciones:

Si el divisor M es cero, el hardware no puede completar la operación y genera una interrupción de división por cero inmediatamente antes de iniciar los ciclos.

División restauradora y no restauradora

Dependiendo de cómo el circuito gestione los residuos parciales negativos, existen dos variantes principales de implementación en hardware.

División restauradora:

Si la resta $A - M$ da un resultado negativo, el divisor no cabe. Por consiguiente, se suma de nuevo M al acumulador ($A \leftarrow A + M$) para restaurar su valor previo y se coloca $Q_n = 0$.

Desventaja: Requiere dos operaciones sobre la ALU en los ciclos donde no cabe.

División no restauradora:

En lugar de restaurar el valor de A de inmediato, se pospone la operación aritmética para el siguiente ciclo aprovechando el desplazamiento.

- Si $A \geq 0$ Desplazar a la izquierda y restar M ($2A - M$)
- Si $A < 0$ Desplazar a la izquierda y sumar M ($2A + M$)

Ventaja: Realiza exactamente una operación de ALU por ciclo de reloj, duplicando la velocidad.

Limitaciones de la coma fija

Los formatos de enteros y de coma fija asignan una cantidad invariable de bits a la parte entera y a la parte fraccionaria, restringiendo severamente su utilidad en cálculo general.

El dilema de la coma fija:

Si fijamos 16 bits para la parte entera y 16 bits para la fracción en una palabra de 32 bits, el valor máximo representable es solo ≈ 65535 , y la menor fracción distinta de cero es $2^{-16} \approx 0.000015$.

Incapacidad de representar escalas dispares:

No permite almacenar en un mismo formato magnitudes astronómicas (masa del sol: 1.98×10^{30} kg) y fenómenos subatómicos (radio de Bohr: 5.29×10^{-11} m).

La solución: Punto flotante

La posición del punto decimal (o la dízima) varía dinámicamente según un exponente, permitiendo intercambiar resolución por rango dinámico según la escala de cada número mediante notación científica binaria:

$$\pm M \times 2^E$$

El estándar IEEE 754

Publicado originalmente en 1985 bajo el liderazgo de William Kahan, el estándar IEEE 754 unificó el formato y el comportamiento aritmético del punto flotante en toda la industria informática.

Distribución de campos en una palabra IEEE 754:



Fórmula de evaluación: $\text{Valor} = (-1)^s \times 2^{e - \text{sesgo}} \times (1.f)_2$

Formatos estándar de 32 y 64 bits

Simple precisión (32 bits)

Tipo `float` en lenguaje C

- • **Signo:** 1 bit (bit 31)
- • **Exponente:** 8 bits (bits 30--23)
- • **Sesgo:** 127 ($2^{8-1} - 1$)
- • **Mantisa:** 23 bits (bits 22--0)
- • **Precisión efectiva:** 24 bits (≈ 7 dígitos decimales)
- • **Rango aproximado:** 10^{-38} a 10^{38}

Doble precisión (64 bits)

Tipo `double` en lenguaje C

- • **Signo:** 1 bit (bit 63)
- • **Exponente:** 11 bits (bits 62--52)
- • **Sesgo:** 1023 ($2^{11-1} - 1$)
- • **Mantisa:** 52 bits (bits 51--0)
- • **Precisión efectiva:** 53 bits (≈ 16 dígitos decimales)
- • **Rango aproximado:** 10^{-308} a 10^{308}

Representación del exponente con sesgo

El exponente en IEEE 754 no se guarda en complemento a dos, sino mediante una representación desplazada por una constante denominada sesgo.

Fórmulas de conversión:

$$e = E + \text{sesgo}$$

$$E = e - \text{sesgo}$$

En 32 bits: sesgo = 127. Si el exponente real es $E = 3$, se almacena $e = 3 + 127 = 130 = 10000010_2$.

¿Por qué usar sesgo en lugar de complemento a dos?

La razón principal es la **eficiencia en la comparación de magnitudes**.

Al usar sesgo, todos los exponentes almacenados son enteros positivos ($e \geq 0$).

Esto permite que el hardware compare el orden de magnitud de dos números usando comparadores estándar de enteros sin signo, sin requerir lógica especial para números negativos.

Normalización y el bit implícito

En el sistema binario, todo número no nulo expresado en notación científica tiene la forma particular:

$$1.b_1b_2b_3\dots b_m \times 2^E$$

Dado que en base 2 el único dígito distinto de cero es el 1, el primer bit antes de la coma siempre es obligatoriamente 1 en cualquier número normalizado.

La ganancia del bit oculto:

Dado que sabemos con total certeza que el bit entero siempre es 1, **el estándar no lo almacena físicamente en la memoria.**

Almacenamos: 23 bits de fracción (f)
Obtenemos: **24 bits de precisión real**

Este principio duplica la resolución efectiva del formato sin gastar silicio adicional.

Valores especiales en el estándar IEEE 754

El estándar reserva combinaciones extremas del campo de exponente ($e = 0$ y $e = e_{\text{máx}}$) para codificar estados especiales del cálculo.

Exponente (e)	Mantisa (f)	Significado	Valor matemático interpretado
$e = 0$	$f = 0$	Cero ($+0$ y -0)	
$e = 0$	$f \neq 0$	Números subnormales	Resultados de operaciones que producen valores demasiado pequeños para ser representados como números normales
$1 \leq e \leq 254$	Cualquiera	Números normalizados	$\pm 2^{e-126} \times$ el número operativo estándar
$e = 255$	$f = 0$	Infinito ($+\infty$ / $-\infty$)	Resultado de división por cero o desbordamiento superior
$e = 255$	$f \neq 0$	NaN	Operaciones indefinidas (0/0, ∞/∞ , $\infty - \infty$)

Errores numéricos y fenómenos de cálculo

Trabajar con punto flotante exige comprender que los números reales en una computadora son aproximaciones discretas sobre una recta numérica continua.

Fracciones periódicas en base 2:

Así como en base 10 la fracción $1/3 = 0.333\dots$ es infinita, en base 2 fracciones comunes como 0.1 y 0.2 se convierten en patrones binarios periódicos infinitos:

$$0.1_{10} = 0.00011001100110011\dots_2$$

Cancelación catastrófica:

Ocurre al restar números casi idénticos en magnitud. Los bits significativos se cancelan a cero y el resultado queda dominado por el redondeo previo.

Desbordamiento superior e inferior:

El desbordamiento superior excede el valor máximo representable ($+\infty$). El inferior ocurre cuando la magnitud cae por debajo del rango subnormal.

Síntesis de la primera sesión

1. Aritmética de enteros en hardware:

La multiplicación se optimiza mediante el algoritmo de Booth evitando conversiones con signo, mientras que la división no restauradora reduce los ciclos de ALU por iteración.

2. Estructura de IEEE 754:

Combina un bit de signo, un exponente con sesgo para facilitar comparaciones y una mantisa con bit 1 implícito para maximizar la resolución.

3. Pregunta detonante para el taller:

Si intentamos sumar 1.0×2^{10} con 1.0×2^{-15} , ¿cómo alinea la CPU los exponentes para poder sumar sus mantisas sin perder precisión?

Semana 04

Sesión 02: Taller práctico

IC3101: Arquitectura de computadores

Objetivos de la segunda sesión

Desarrollar destrezas prácticas de conversión, decodificación y cálculo aritmético:

- **Conversión decimal a IEEE 754:** Dominar el procedimiento sistemático paso a paso en formatos de 32 y 64 bits.
- **Decodificación inversa:** Transformar patrones binarios y palabras hexadecimales reales a valores decimales legibles.
- **Aritmética de punto flotante:** Ejecutar sumas y restas manuales mediante la alineación de exponentes y normalización de mantisas.
- **Fenómenos en software:** Identificar fuentes de error por redondeo en lenguaje C y aplicar comparaciones seguras con ϵ .
- **Ejercicios de práctica:** Resolver problemas prácticos y trazas de hardware para afianzar el aprendizaje autónomo.

Metodología de conversión decimal a IEEE 754

Procedimiento estándar en cuatro fases para codificar una magnitud decimal en simple precisión (32 bits):

1 Bit de signo

Identificar la polaridad de la magnitud: $s = 0$ si es positivo, $s = 1$ si es negativo.

2 Magnitud binaria

Convertir la parte entera por división entre 2 y la fracción por multiplicación sucesiva hasta fijar la coma.

3 Normalización

Desplazar la coma para obtener la forma $1.f \times 2^E$ y aislar el exponente real E .

4 Empaquetado

Calcular el sesgo $e = E + 127$, rellenar la mantisa a 23 bits y convertir a hexadecimal.

Decimal

→

Signo (s)

→

Coma fija

→

Forma $1.f \times 2^E$

→

Palabra IEEE 754

Ejemplo guiado 1: Número con fracción exacta

Convertir el número decimal **+26.625** a formato IEEE 754 de simple precisión (32 bits).

1. Signo y parte entera:

$s = 0$ (positivo)

$26 \div 2 = 13$ (residuo 0)

$13 \div 2 = 6$ (residuo 1)

$6 \div 2 = 3$ (residuo 0)

$3 \div 2 = 1$ (residuo 1) $\rightarrow 1 \div 2 = 0$ (residuo 1)

$26_{10} = 11010_2$

2. Parte fraccionaria:

$0.625 \times 2 = 1.25 \rightarrow$ entero 1

$0.25 \times 2 = 0.50 \rightarrow$ entero 0

$0.50 \times 2 = 1.00 \rightarrow$ entero 1

$0.625_{10} = 0.101_2$

Número en coma fija: 11010.101_2

3. Normalización y empaquetado:

Normalizado: $11010.101_2 = 1.1010101_2 \times 2^4 \Rightarrow E = 4$

Exponente con sesgo: $e = 4 + 127 = 131_{10} = 10000011_2$

Mantisa (23 bits): $10101010000000000000000_2$

Binario: 0 10000011 10101010000000000000000 $\rightarrow$ **Hex:** 0x41D54000

Ejemplo guiado 2: Fracción periódica y truncamiento

Convertir el número decimal **-13.1** a formato IEEE 754 de 32 bits y analizar la pérdida por redondeo.

Signo y parte entera:

$s = 1$ (negativo)

$13_{10} = 1101_2$

Fracción decimal recurrente:

$0.1 \times 2 = 0.2 \rightarrow 0 \mid 0.2 \times 2 = 0.4 \rightarrow 0$

$0.4 \times 2 = 0.8 \rightarrow 0 \mid 0.8 \times 2 = 1.6 \rightarrow 1$

$0.6 \times 2 = 1.2 \rightarrow 1 \mid 0.2 \times 2 = 0.4 \rightarrow 0 \dots$

$0.1_{10} = 0.00011_2$ (Periódico)

Normalización a 23 bits:

$1101.0001100110011\dots_2$

Normalizado: $1.10100011001100110011001\dots \times 2^3$

$E = 3 \rightarrow e = 3 + 127 = 130 = 10000010_2$

Bit siguiente al corte es 1, aplicando redondeo al par más cercano:

$f = 10100011001100110011010_2$

Representación final: $1 \ 10000010 \ 10100011001100110011010 \rightarrow$ Hex: **0xC151999A**

Decodificación inversa: De hexadecimal a decimal

Determinar el valor decimal real codificado en la palabra hexadecimal IEEE 754 de 32 bits: **0xC24C0000**.

Paso 1: Expansión a 32 bits binarios

11000010010011000000000000000000

Signo: $s = 1 \rightarrow$ Negativo

Exponente: $e = 132_{10}$

Exp. real: $E = 132 - 127 = 5$

Paso 2: Reconstrucción del número con bit 1 implícito

Significando completo: $1.f = 1.10011000..._2$

Multiplicación por 2^E : Desplazar la coma 5 posiciones a la derecha:

$110011.0_2 = 32 + 16 + 2 + 1 = 51_{10}$

Valor decimal final: **-51.0**

Aritmética de punto flotante: Suma y resta

A diferencia de los enteros, operar dos números en punto flotante exige un procedimiento en cuatro etapas de hardware en la FPU:

1 Alinear exponentes

Restar exponentes ($e_1 - e_2$) y desplazar a la derecha la mantisa del menor hasta igualar escalas.

2 Operar mantisas

Sumar o restar algebraicamente los significandos alineados considerando sus signos de polaridad.

3 Normalizar

Reajustar la coma a la forma $1.f$ y compensar el exponente tras posibles desbordamientos o cancelaciones.

4 Redondear

Ajustar la mantisa a los 23 bits definitivos y verificar excepciones de desbordamiento ($+\infty$ / underflow).

Operandos (A, B)

→

Alineación ($e_1 = e_2$)

→

Suma de mantisas

→

Normalización

→

Resultado final

Traza práctica de suma en punto flotante

Sumar en punto flotante: $A = 1.000_2 \times 2^3$ (8.0) y $B = 1.100_2 \times 2^1$ (3.0).

1. Alinear

Diferencia: $3 - 1 = 2$

Desplazar B 2 bits:

$$B = 0.011_2 \times 2^3$$

2. Sumar

Mantisa A : 1.000_2

+ Mantisa B : 0.011_2

$$= \text{Suma: } 1.011_2$$

3. Normalizar

Forma ya normalizada:

$$1.011_2 \times 2^3$$

4. Resultado

$$1.011_2 \times 2^3$$

$$1011.0_2 = 11.0_{10}$$

$$\text{Exacto: } 8 + 3 = 11$$

Peligro de absorción: Si la diferencia de exponentes es mayor a 24 en simple precisión, la mantisa del sumando menor se desplazará más de 23 posiciones hacia la derecha, expulsando todos sus bits significativos. En tal caso, $A + B = A$, absorbiendo el número menor como si fuera cero.

Fenómenos en software: La ilusión de los flotantes

Las particularidades del hardware de punto flotante impactan directamente el desarrollo de software en lenguajes como C, C++ o Python.

// El error clásico en C:

```
float a = 0.1f;
float b = 0.2f;
if (a + b == 0.3f) {
    printf("Iguales\n");
} else {
    printf("Distintos: %.9f\n", a + b);
}
// Salida real: Distintos: 0.300000012
```

Comparación segura mediante épsilon (ϵ):

En lugar de usar el operador de igualdad `==` con números de punto flotante, es más seguro verificar si la diferencia absoluta es menor que una tolerancia mínima admisible.

```
#define EPSILON 1e-6
if (fabs(a + b - 0.3f) < EPSILON) ...
```

Acumulación de error en ciclos:

Sumar `0.1f` cien millones de veces en un ciclo no produce exactamente un millón, sino un valor desviado debido al redondeo en cada iteración.

Taller en vivo: Ejercicios para resolver en parejas

Formen parejas y resuelvan en papel los siguientes dos ejercicios de aplicación práctica en los próximos 10 minutos.

Reto A: Conversión a IEEE 754

Convertir el número decimal **-45.75** a formato IEEE 754 de simple precisión (32 bits).

- Determinar signo, binario entero y fraccionario.
- Normalizar y calcular exponente sesgado.
- Expresar el resultado final en hexadecimal.

Reto B: Decodificación inversa

Decodificar la siguiente palabra hexadecimal de 32 bits a su valor decimal real: **0x42E80000**.

- Desglosar campos de signo, exponente y mantisa.
- Calcular exponente real $E = e - 127$.
- Reconstruir el significando con el 1 implícito.

Levanten la mano si tienen dudas intermedias. En breve revisaremos la solución detallada de ambos problemas en la pizarra.

Solución detallada de los ejercicios de taller

Solución Reto A: -45.75

- Signo: $s = 1$ (negativo)
- Entero: $45 = 101101_2$
- Fracción: $0.75 = 0.11_2$
- Coma fija: 101101.11_2
- Normalizado: $1.0110111_2 \times 2^5 \Rightarrow E = 5$
- Exponente: $e = 5 + 127 = 132 = 10000100_2$
- Mantisa: 0110111000000000000000_2

Hexadecimal: 0xC236C000

Solución Reto B: 0x42E80000

- Binario: 0 10000101 11010000000...
- Signo: $s = 0$ (positivo)
- Exponente: $e = 133_{10} \Rightarrow E = 133 - 127 = 6$
- Significando: 1.1101_2
- Desplazamiento por 2^6 : 1110100.0_2
- Cálculo: $64 + 32 + 16 + 4 = 116_{10}$

Decimal real: +116.0

Ejercicios de práctica recomendados

Problemas seleccionados para consolidar el dominio del estándar IEEE 754 y la aritmética en hardware mediante trabajo autónomo:

1. Conversiones y decodificaciones:

- • **Ejercicio A:** Convertir -85.125_{10} a formato IEEE 754 de simple precisión (32 bits).
- • **Ejercicio B:** Decodificar el patrón hexadecimal `0x41E40000` a su valor decimal.
- • **Ejercicio C:** Representar $+0.05_{10}$ en 32 bits identificando su fracción periódica y redondeo.

2. Aritmética y trazas en hardware:

- • **Ejercicio D:** Sumar en 32 bits $A = 1.010_2 \times 2^4$ y $B = 1.100_2 \times 2^2$ con las 4 etapas.
- • **Ejercicio E:** Multiplicar $(-6) \times (-5)$ en palabras de 4 bits mediante el algoritmo de Booth.
- • **Ejercicio F:** Explicar si ocurre absorción al sumar 1.0×2^{30} con 1.0×2^5 en 32 bits.

Recomendación metodológica: Resolver estos ejercicios paso a paso en papel antes de verificar sus resultados con una calculadora binaria o código en C.

Quiz formativo interactivo

Evaluemos los conceptos clave de la jornada contestando mentalmente las siguientes tres preguntas:

Pregunta 1:

En el algoritmo de Booth, si la pareja de bits $[Q_0, Q_{-1}]$ evaluada es 10, ¿qué operación realiza la ALU?

A) $A \leftarrow A + M$ | B) $A \leftarrow A - M$ | C) Ninguna operación aritmética

Pregunta 2:

¿Cuál es el valor del sesgo utilizado en el formato IEEE 754 de simple precisión (32 bits)?

A) 128 | B) 127 | C) 1023

Pregunta 3:

¿Qué estado especial representa una palabra con todos los bits del exponente en 1 y mantisa distinta de cero?

A) Infinito | B) Cero negativo | C) NaN

Respuestas comentadas del quiz

Respuesta 1: B) $A \leftarrow A - M$

Inicio de bloque

La transición 10 marca el inicio de una secuencia de unos consecutivos en el multiplicador, lo que exige restar el multiplicando del acumulador antes del desplazamiento aritmético.

Respuesta 2: B) 127

Sesgo de 8 bits

Para un campo de 8 bits de exponente, el sesgo estándar corresponde a $2^{8-1} - 1 = 128 - 1 = 127$, permitiendo representar exponentes reales entre -126 y +127.

Respuesta 3: C) NaN

Indeterminación

El exponente máximo con mantisa no nula codifica una indeterminación matemática producida por operaciones inválidas, preservando la estabilidad del procesador.

Conclusiones y preparación para la Semana 05

Hemos culminado con éxito el bloque fundacional de sistemas numéricos y aritmética digital de bajo nivel (Semanas 01 a 04).

Próxima sesión: Semana 05

Introducción al lenguaje C y ensamblador x86

- Estructura básica de un programa en lenguaje C y en NASM
- Banco de registros x86: RAX, RBX, RCX, RDX, RSI, RDI, RSP, RBP
- Primeras instrucciones de transferencia y aritmética: `mov`, `add`, `sub`
- Depuración interactiva paso a paso con GDB

¡Muchos éxitos con los ejercicios de práctica y nos vemos la próxima semana!